Prompt Engineering for Tabular Prediction using LLMs

Purpose – In many real-world applications, Large Language Models (LLMs) are used to reason over structured tabular data rather than plain text. Examples include risk assessment, customer churn prediction, and medical triage systems.

This assignment investigates whether advanced prompt engineering strategies improve LLM performance when making predictions from structured data.

By completing this assignment, you will learn how to:

- use LLMs for tabular reasoning tasks
- design and compare prompt engineering techniques
- evaluate predictions using classification metrics
- analyze where LLM reasoning fails

Task - You will build an LLM-based prediction system that predicts whether a passenger survived the Titanic disaster.

Prediction target: Survived

1 = survived & 0 = did not survive

Use two LLMs released in 2025 (for example): GPT-4.1 and Claude Sonnet 4

- Apply the following prompting strategies for each model:

  - Zero-shot prompting

  - Five-shot prompting

  - Chain-of-thought prompting (5-shot)

  - Self-consistency prompting (5-shot reasoning)

  - Tree-of-Thought prompting

- This results in 10 experimental conditions (2 models × 5 prompting methods).

Dataset – Use the Titanic dataset from Kaggle: Use minimum 500 samples as test set for final evaluation. Use the remaining data to select few-shot examples and refine prompts – Do NOT include test examples in prompts or development.

Input Format – Each passenger should be presented to the model in a structured format (natural language format).

Required Output Format and Experimental Results – For every prediction, return a structured output containing: label (0/1). For each model and prompting strategy, run predictions on the test set and compute the following metrics:

LLM	Prompting Method	Accuracy	Precision	Recall	F1

Hard Case Analysis – Identify at least 10 passengers that the models misclassify**.** Analyze possible causes such as:

• missing information
• conflicting features
• model reasoning errors

Grading Rubric: Prompt Engineering for Tabular Prediction using LLMs

Criterion	Description	Points
Prompt Design, Setup & Implementation	Implemented required prompting strategies (Zero-shot, 5-shot, CoT, Self-consistency, ToT) for two LLMs. Titanic dataset used correctly with test set ≥500 and no prompt tuning on test data.	3.5
Evaluation & Results	Predictions generated for all experiments with metrics (Accuracy, Precision, Recall, F1) reported in a clear table.	1.5
Hard Case Analysis	At least 10 misclassified passengers analyzed with reasonable explanations.	1

Enterprise Retrieval-Augmented Generation (RAG) with WixQA

Purpose

Apply advanced techniques for building and evaluating Retrieval-Augmented Generation systems in enterprise environments. You must construct a production-style RAG pipeline using the WixQA benchmark, which contains a real enterprise knowledge base and customer-support queries. The goal is to analyze how retrieval quality, chunking strategy, reranking, and query rewriting influence the factuality and completeness of generated answers.

Dataset Configuration

Knowledge Source: 6,221 Wix Help Center articles.
Development Dataset: WixQA-Synthetic.
Evaluation Datasets: WixQA-ExpertWritten and WixQA-Simulated.

WixQA-Synthetic queries are used to tune the end-to-end RAG system configuration (e.g., retrieval size, chunking strategy). Final evaluation must be performed only on WixQA-ExpertWritten and WixQA-Simulated.

```
Knowledge Base
(6,221 articles)
        |
        ▼
Build RAG system
        |
        ▼
Development queries
(WixQA-Synthetic – 6,221)
        |
Tune system parameters
        |
        ▼
Final pipeline configuration
        |
        ▼
Evaluation queries
(WixQA-ExpertWritten + Simulated)
        |
        ▼
Report final metrics
```

Part 1 — Baseline Enterprise RAG System

```
User Query
   ↓
Query Embedding
   ↓
Retriever (Vector Search)
   ↓
Top-k Evidence Chunks
   ↓
LLM Generator → Grounded Answer
```

```
Enterprise Knowledge Base
(6,221 Wix Help Center Articles)
          ↓
       Retriever
```

Apply a baseline RAG pipeline using the WixQA knowledge base. You must: (1) chunk the 6,221 knowledge-base articles, (2) compute embeddings for all chunks and index them in a vector store, (3) encode each query, (4) retrieve the top-k chunks, and (5) generate an answer using only the retrieved evidence.

Use WixQA-Synthetic for development and system tuning. Use WixQA-ExpertWritten and WixQA-Simulated only for evaluation.

Record the baseline system configuration in the following table.

Component	Configuration Used
Chunk size / overlap	
Embedding model	
Vector database	
Retriever type	
Generator model	
Prompting strategy	

You must evaluate multiple retrieval configurations using the development queries (WixQA-Synthetic) in order to determine the optimal retrieval settings, chunk size: {300, 500}, Overlap = {30, 60} and k = {1, 3, 5, 10} For each configuration, compute the following evaluation metrics:

- Context Recall – measures whether the retrieved context contains the necessary information to answer the question. You can use LLM as a judge with temperature 0. For each query, provide the query, the ground-truth answer, and the retrieved context to the LLM judge to determine whether the retrieved context contains sufficient information to answer the query correctly by returning binary answer: Yes or NO.
- F1 Score – measures token overlap between the generated answer and the ground-truth answer.
- ROUGE-1 – measures unigram overlaps and content coverage.

After running the experiments, analyze the results and determine the optimal hyperparameter configuration. You must complete and submit the following table as part of your report.

Chunk Size	**Overlap**	**k**	**Context Recall**	**F1**	**ROUGE-1**	**Observation**
300	30	1				
..	..	.				

After completing the experiments, you must select the optimal hyperparameter configuration based on the experimental results.

You should consider both:

- Retrieval quality (Context Recall)
- Answer quality (F1 and ROUGE-1)

The selected configuration must balance high retrieval coverage and strong answer generation performance.

Part 2 — Retrieval Evaluation

Evaluate retrieval quality using Context Recall. Context Recall measures whether the retrieved context contains the essential information required to produce the ground-truth answer.

Apply this evaluation on WixQA-ExpertWritten and WixQA-Simulated. Record retrieval performance using the following table.

Dataset	Optimal Chunk size	Optimal Overlap	Optimal k	Context Recall	Brief Observation
WixQA-ExpertWritten					
WixQA-Simulated					

In the Observation column, briefly describe the retrieval behavior of the system (for example, whether the retrieved evidence consistently contains the necessary information or whether certain queries still fail to retrieve relevant context).

Part 3 — Generation Evaluation

Evaluate generated answers using four complementary metrics: F1, ROUGE-1, ROUGE-2, and Factuality. Apply F1 to measure token-level information overlap, ROUGE-1 and ROUGE-2 to measure coverage of important content and procedural steps, and Factuality to determine whether the generated answer preserves the essential meaning of the ground-truth answer. Factuality should be evaluated using an LLM judge that compares generated answer to ground-truth answer and assigns 1 if the generated answer preserves the essential meaning without contradiction, else 0.

Use the following interpretation when analyzing answer quality: a strong system should score well on overlap metrics and remain factually faithful even when retrieved context becomes noisy.

Use only the configuration selected on WixQA-Synthetic; do not retune on WixQA-ExpertWritten or WixQA-Simulated.

Record generation quality using the following table.

Dataset	F1	ROUGE-1	ROUGE-2	Factuality

WixQA-ExpertWritten				
WixQA-Simulated				

Part 4 — System Improvements

Apply the following three improvements and compare each one against the baseline system: semantic chunking, reranking, and query rewriting. For each improvement, keep the generator, embedding model, vector store, prompt, and all untargeted hyperparameters fixed to the optimal baseline configuration.

Semantic Chunking: replace fixed-size chunking with semantically coherent chunk boundaries based on sentence embedding similarity.

```
Knowledge Base → Semantic Chunking → Vector Index
                                  |
                                  ▼
User Query → Query Embedding → Retriever → Top-k Evidence → LLM Generator →
Grounded Answer
```

Reranking: apply two-stage retrieval, first retrieve top-50 candidates as a fixed design choice, then rerank and keep the same final k chosen in Part 1 for reranking them using a cross-encoder or an LLM-based scoring. retrieving the top-50 candidate chunks and then reranking them using a cross-encoder or an LLM-based scoring model before selecting top-k.

```
Knowledge Base → Document Chunking → Document Embedding → Vector Index
                                  |
                                  ▼
User Query → Query Embedding → Retriever → Top-N Candidates → Reranker → Top-k
Evidence → LLM Generator → Grounded Answer
```

Query Rewriting: rewrite the original user query into a retrieval-oriented query before search.

```
User Query → Query Rewriting → Query Embedding → Retriever → Top-k Evidence → LLM
Generator → Grounded Answer
```

Use the following metrics for comparative analysis: Context Recall, F1, ROUGE-1, ROUGE-2, and Factuality. These metrics jointly evaluate whether an improvement helps retrieval, preserves procedural content, and reduces hallucinations.

Record the comparative analysis using the following table.

System	Dataset	Context Recall	F1	ROUGE-1	ROUGE-2	Factuality
Baseline RAG	WixQA-ExpertWritten					
Baseline RAG	WixQA-Simulated					
+ Semantic Chunking	WixQA-ExpertWritten					
+ Semantic Chunking	WixQA-Simulated					
+ Reranking	WixQA-ExpertWritten					
+ Reranking	WixQA-Simulated					
+ Query Rewriting	WixQA-ExpertWritten					
+ Query Rewriting	WixQA-Simulated					

Criteria for Success

Apply a complete and reproducible enterprise RAG evaluation. A strong submission will present a correct baseline system, a clear retrieval-noise study, a comparative analysis of the three improvements, well-formed tables and plots, and concise technical interpretation of system behavior.

Grading Rubric (20 Points)

Component	Points	Description
Baseline RAG implementation	4	Correct end-to-end system with indexed corpus, retrieval, and grounded generation
Retrieval evaluation	2	Correct Context Recall evaluation and clear interpretation
Generation evaluation	2	Correct use of F1, ROUGE-1, ROUGE-2, and Factuality
System improvements	3 * 2 = 6	Correct implementation and comparison of semantic chunking, reranking, and query rewriting

You are required to follow:

You must submit both source code and **a single written report (MS Word or PDF).** Submissions that do not follow this format will receive a penalty.

1. Source Code

- Python (Jupyter Notebook): Must be well-organized, properly indented, and include clear comments
- The notebook should run from start to finish without manual intervention

2. Written Report (MS Word or PDF)

Your report must be a single consolidated document and include the following:

1. Summary of each model and Mention hyperparameters used
2. Outputs and results: Include tables summarizing performance metrics (e.g., Accuracy, Precision, Recall, F1, Perplexity, etc. as applicable)
3. Screenshots: You must include screenshots to verify that your code actually ran:
    - Training logs showing first few epochs
    - Training logs showing completion of all epochs
    - Screenshots of important outputs such as:
        - Confusion matrices
        - Learning curves (loss/accuracy plots)
        - Classification reports

3. Submission Format

- One Jupyter Notebook (.ipynb) containing all source code
- One report (.docx or .pdf) with results, tables, screenshots, and analysis
- Both files must be clearly named:
    - HW1_LastName_FirstName.ipynb
    - HW1_LastName_FirstName.pdf

- **Failure to submit the source code will result in a deduction of full/partial points.**
- Before submitting the source code, please double-check that it runs without any errors.
- Must submit the files separately.
- Do not compress into a zip file.
- HW submitted more than 24 hours late will not be accepted for credit.